A Configurable Mixed-Precision Fused Dot Product Unit for GPGPU Tensor Computation

Nikhil Rout
Vellore Institute of Technology, Chennai
Hyderabad, India
nikhilrout97@gmail.com

Blaise Tine
University of California, Los Angeles
Los Angeles, USA
blaisetine@cs.ucla.edu

Abstract

Efficient mixed-precision MMA operations are critical for accelerating Deep Learning workloads on GPGPUs. However, existing open-source RTL implementations of inner dot products rely on discrete arithmetic units, leading to suboptimal throughput and poor resource utilization. To address these challenges, we propose a scalable mixed-precision dot product unit that integrates floating-point and integer arithmetic pipelines within a singular fused architecture, implemented as part of the open-source RISC-V based Vortex GPGPU's Tensor Core Unit extension. Our design supports low-precision multiplication in (FP16/BF16/FP8/BF8/INT8/UINT4) formats and higher-precision accumulation in (FP32/INT32), with an extensible framework for adding and evaluating other custom representations in the future. Experimental results demonstrate 4-cycle operation latency at 306.6 MHz clock frequency on the AMD Xilinx Alveo U55C FPGA, delivering an ideal filled pipeline throughput of 9.812 GFLOPS in a 4-thread per warp configuration.

CCS Concepts

• **Computer systems organization** → *Multicore architectures.*

Keywords

GPGPU, Microarchitecture, Mixed-Precision, Fused Dot Product

1 Introduction

Microbenchmarking NVIDIA Volta Architecture [12] Warp-Matrix-Multiply-Accumulate (WMMA) instructions have demonstrated that each "Tensor Core" is essentially a grid of 16 mixed-precision Four-Element Dot Product (FEDP) units scheduled in parallel, completing a 4×4×4 matrix multiply accumulate per cycle in a 4-stage pipeline [7, 13]. Fig. 1 illustrates how we adopt a 2×2 grid of FEDP units to form a Tensor Core Unit (TCU) within a Vortex GPGPU [16] sub-core in a 4-thread per warp scheduler configuration. To address the lack of high-performance mixed-precision fused dot product implementations in the open-source GPGPU design space:

- We propose a configurable 4-stage fused dot product architecture supporting low-precision (FP16/BF16/FP8/BF8) multiplication with FP32 accumulation; implemented as part of the Vortex GPGPU's TCU extension [15]
- We describe a unified pipeline methodology for integrating integer arithmetic within the floating-point datapath, incurring minimal overhead by maximizing resource reuse
- We demonstrate ~3.7x performance improvements over an equivalent Berkeley HardFloat [6] based implementation, achieving 306.6 MHz operational F_{max} and 2.453 GFLOPS single-cycle throughput at less than 60% the area cost

2 Mixed-Precision Floating-Point Datapath

2.1 Key Arithmetic Submodules

The floating-point dot product pipeline requires several multi-operand additions. Initially, we adopted a 2-operand adder reduction tree that exhibited high latency from accumulated carry propagation delays. **Carry-Save Adders (CSAs)** proved optimal for reducing multiple operands without prior carry dependencies. The CSA uses recursively generated 4:2 compressors, with 3:2 compressors (Full Adders) fallback handling odd operand count cases, before final summation via **Kogge-Stone Adders (KSAs)**. KSAs outperform Carry-Look-Ahead designs by sacrificing area efficiency to achieve lower fanout at every stage due to their parallel prefix tree structures. We enhance the base CSA with a parallel MOD-4 operand grouping structure to minimize critical path delay for large operand counts.. We also implement **Wallace Tree Multipliers** whose partial products are reduced effectively using CSA structures. We decide against incorporating Radix-4 booth recoding in our design, since the bit-pair encoding overhead outweighs the benefit of halving partial products at our target 4-11 bit widths.

2.2 Shared Low-Precision Multiplication

The first pipeline stage performs low-precision multiplication in parallel with finding the maximum exponent. Inputs are packed as FP16/BF16 pairs or FP8/BF8 quads per 32-bit register. Class-wise shared Wallace tree multipliers process full mantissas (including implicit bits) for each dot product element. FP8/BF8 require an additional multiplication and summation to maintain pipeline consistency. Finally, format selection multiplexers route the results to generate raw E8M25 intermediates for accumulation.

2.3 Maximum Exponent and Alignment

Exponent addition and FP32 exponent bias conversion (as required) are performed according to the equation:

$$EXP_{FP32} = EXP_A + EXP_B + BIAS_{FP32} - (2 \times BIAS_{FP16}) + 1$$

Maximum exponent identification builds upon a subtractor-based comparator architecture [14] computing all N×N pairwise exponent differences concurrently. Each comparison generates a sign bit indicating relative magnitude, forming a sign matrix where the maximum exponent corresponds to the column containing all zeros.The maximum exponent index is determined by performing reduction-OR over each column and inverting the result, producing a one-hot selection vector. This approach provides O(1) depth versus traditional reduction tree comparator schemes, while also calculating shift amounts by simply negating the pre-computed differences. Product significands are then aligned using these shift amounts and sign-extended in the second pipeline stage.

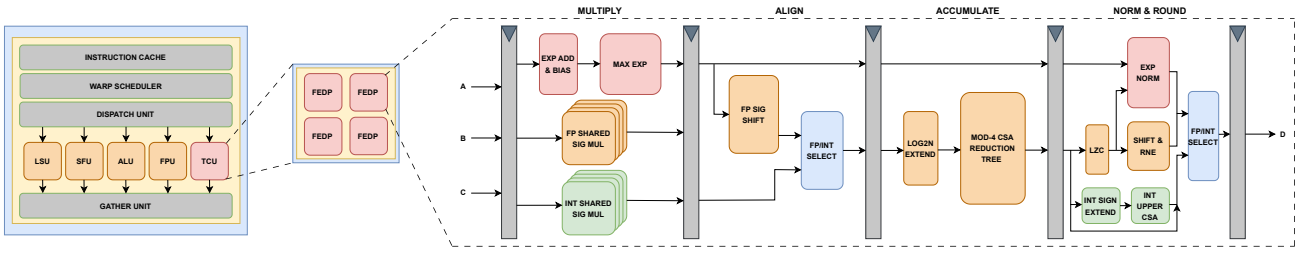

Figure 1: GPGPU Mixed-Precision Fused Dot Product Unit 4-Stage Pipeline Microarchitecture

2.4 Accumulation, Normalization and Rounding

Unlike traditional approaches that accumulate the addend "C" after dot product summation – incurring additional alignment, normalization, and rounding overhead – we process the addend's exponent and significand alongside product terms from the first stage. The aligned significands are then accumulated via a $25 + \log_2(N)$-bit MOD-4 CSA in the third pipeline stage. Finally, standard LZC normalization and RNE rounding are performed in the last stage.

3 Fusing the Integer Pipeline

Integer dot product operations require multiple arithmetic components already present in the FP datapath [3, 4]. Fusing both pipelines eliminates the need for an arbiter and scheduling two separate execution units via the same interface. We support INT8 and UINT4 multiplication with INT32 accumulation, where products are sign-extended to 25 bits for accumulator compatibility. Rather than forwarding the complete 32-bit addend C to the final stage, we employ a novel splitting strategy that partitions C addition into two components. The lower 25 bits are summed in the existing third stage MOD-4 CSA accumulator, while only the upper 7 bits and product sign bits propagate through the pipeline; hence reducing intermediate register overhead. The final integer result's upper 7 bits are calculated in parallel with FP normalization and rounding, before concatenation with the lower 25-bit accumulation result.

4 Evaluation

We evaluate our FEDP design against equivalent Xilinx DSP IP and Berkeley HardFloat [6] based discrete implementations, targeting 300MHz clock frequency on the AMD Xilinx Alveo U55C FPGA across a range of threads/warp configurations (N = 4, 8, 16, 32).

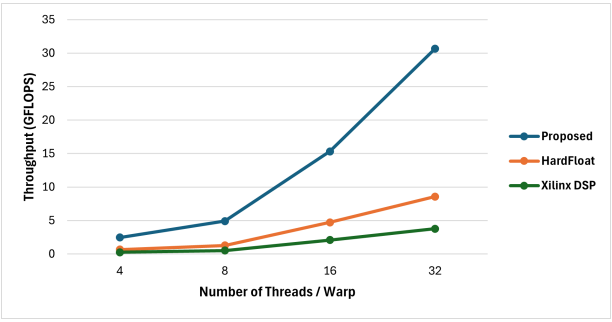

Figure 2: FEDP Backends Performance Scaling (FP16/BF16)

Fig. 2 shows that our proposed architecture achieves significantly higher single-cycle throughput[1] scaling (2.453–30.662 GFLOPS) compared to HardFloat (0.658–8.575 GFLOPS, ~3.7x) and Xilinx DSP (0.254–3.768 GFLOPS, ~8.1x). This improvement stems from our 4-cycle latency versus HardFloat's 13 cycles and Xilinx DSP's 42 cycles, and the MOD-4 CSA accumulator structure.

Table 1: FEDP Backends Area Costs (FP16/BF16)

	Backend	N = 4	N = 8	N = 16	N = 32
LUTs	Xilinx DSP	6216	12414	49236	98581
	HardFloat	18400	37002	144001	291207
	Proposed	**10945**	**21899**	**95336**	**188077**
FFs	Xilinx DSP	9107	18063	70738	141314
	HardFloat	6163	12153	46850	93190
	Proposed	**2364**	**4624**	**14967**	**29769**
DSPs	Xilinx DSP	64	128	512	1024
	HardFloat	16	32	128	256
	Proposed	**0**	**0**	**0**	**0**

Tab. 1 demonstrates that our design achieves 40-55% LUT reduction versus HardFloat, while being comparable to Xilinx DSP. Flip-Flop usage is significantly reduced by 62-68% compared to HardFloat and 74-79% compared to Xilinx DSP. Additionally, our design eliminates DSP block usage entirely, whereas both baselines require DSP blocks that scale linearly with thread count.

5 Related Work

Although numerous efforts have optimized mixed-precision Fused-Multiply-Add and Dot Product units [2, 9, 14, 17], many active academic projects involving transprecision computing, including Gemmini [5], Virgo [8], and Rocket Chip [1], still rely on Berkeley HardFloat [6]. Similarly, another Tensor Core implementation integrated into the Vortex platform [11] utilized FPnew [10]. These discrete approaches suffer from high latency, accumulated rounding errors, and poor area efficiency compared to our fused architecture.

6 Conclusion

This work presents a high-performance mixed-precision fused dot product unit microarchitecture. Future work includes exploring sparse-gated FEDP and OCP Microscaling Formats (MX) support.

[1] Single-cycle Throughput = (FLOPs / Latency) × F_{max})

References

[1] Krste Asanović, Rimas Avizienis, Jonathan Bachrach, Scott Beamer, David Biancolin, Christopher Celio, Henry Cook, Daniel Dabbelt, John Hauser, Adam Izraelevitz, Sagar Karandikar, Ben Keller, Donggyu Kim, John Koenig, Yunsup Lee, Eric Love, Martin Maas, Albert Magyar, Howard Mao, Miquel Moreto, Albert Ou, David A. Patterson, Brian Richards, Colin Schmidt, Stephen Twigg, Huy Vo, and Andrew Waterman. 2016. *The Rocket Chip Generator.* Technical Report UCB/EECS-2016-17. http://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-17.html

[2] Luca Bertaccini, Gianna Paulin, Matheus Cavalcante, Tim Fischer, Stefan Mach, and Luca Benini. 2024. MiniFloats on RISC-V Cores: ISA Extensions With Mixed-Precision Short Dot Products. *IEEE Transactions on Emerging Topics in Computing* 12, 4 (2024), 1040–1055. doi:10.1109/TETC.2024.3365354

[3] Tom M. Bruintjes, Karel H. G. Walters, Sabih H. Gerez, Bert Molenkamp, and Gerard J. M. Smit. 2012. Sabrewing: A lightweight architecture for combined floating-point and integer arithmetic. *ACM Trans. Archit. Code Optim.* 8, 4, Article 41 (Jan. 2012), 22 pages. doi:10.1145/2086696.2086720

[4] Stef Cuyckens, Xiaoling Yi, Nitish Satya Murthy, Chao Fang, and Marian Verhelst. 2025. Efficient Precision-Scalable Hardware for Microscaling (MX) Processing in Robotics Learning. arXiv:2505.22404 [cs.AR] https://arxiv.org/abs/2505.22404

[5] Hasan Genc, Seah Kim, Alon Amid, Ameer Haj-Ali, Vighnesh Iyer, Pranav Prakash, Jerry Zhao, Daniel Grubb, Harrison Liew, Howard Mao, Albert Ou, Colin Schmidt, Samuel Steffl, John Wright, Ion Stoica, Jonathan Ragan-Kelley, Krste Asanovic, Borivoje Nikolic, and Yakun Sophia Shao. 2021. Gemmini: Enabling Systematic Deep-Learning Architecture Evaluation via Full-Stack Integration. In *Proceedings of the 58th Annual Design Automation Conference (DAC).*

[6] John R. Hauser. 2019. Berkeley HardFloat Floating-Point Arithmetic Package, Release 1. https://www.jhauser.us/arithmetic/HardFloat.html. Accessed: September 5, 2025.

[7] Zhe Jia, Marco Maggioni, Benjamin Staiger, and Daniele P. Scarpazza. 2018. Dissecting the NVIDIA Volta GPU Architecture via Microbenchmarking. arXiv:1804.06826 [cs.DC] https://arxiv.org/abs/1804.06826

[8] Hansung Kim, Ruohan Richard Yan, Joshua You, Tieliang Vamber Yang, and Yakun Sophia Shao. 2025. Virgo: Cluster-level Matrix Unit Integration in GPUs for Scalability and Energy Efficiency. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Rotterdam, Netherlands) *(ASPLOS '25).* Association for Computing Machinery, New York, NY, USA, 1382–1399. doi:10.1145/3676641.3716281

[9] Qiong Li, Chao Fang, and Zhongfeng Wang. 2023. PDPU: An Open-Source Posit Dot-Product Unit for Deep Learning Applications. In *2023 IEEE International Symposium on Circuits and Systems (ISCAS).* IEEE, USA, 1–5. doi:10.1109/ISCAS46773.2023.10182007

[10] Stefan Mach, Fabian Schuiki, Florian Zaruba, and Luca Benini. 2020. Fpnew: An open-source multiformat floating-point unit architecture for energy-proportional transprecision computing. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 29, 4 (2020), 774–787.

[11] Abubakr Nada, Giuseppe Maria Sarda, and Erwan Lenormand. 2025. Cooperative Warp Execution in Tensor Core for RISC-V GPGPU. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA).* 1422–1436. doi:10.1109/HPCA61900.2025.00107

[12] NVIDIA Corporation. 2017. *NVIDIA Tesla V100 GPU Architecture.* Technical Report. https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf

[13] Md Aamir Raihan, Negar Goli, and Tor M. Aamodt. 2019. Modeling Deep Learning Accelerator Enabled GPUs. In *2019 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS).* 79–92. doi:10.1109/ISPASS.2019.00016

[14] Jongwook Sohn and Earl E. Swartzlander. 2016. A Fused Floating-Point Four-Term Dot Product Unit. *IEEE Transactions on Circuits and Systems I: Regular Papers* 63, 3 (2016), 370–378. doi:10.1109/TCSI.2016.2525042

[15] Blaise Tine and Nikhil Rout. 2025. Vortex GPGPU Tensor Core Unit Extension FEDP DRL RTL Backend. https://github.com/vortexgpgpu/vortex/tree/bug_fixes/hw/rtl/tcu/drl.

[16] Blaise Tine, Krishna Praveen Yalamarthy, Fares Elsabbagh, and Kim Hyesoon. 2021. Vortex: Extending the RISC-V ISA for GPGPU and 3D-Graphics. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture* (Virtual Event, Greece) *(MICRO '21).* Association for Computing Machinery, New York, NY, USA, 754–766. doi:10.1145/3466752.3480128

[17] Hao Zhang, Dongdong Chen, and Seok-Bum Ko. 2019. Efficient Multiple-Precision Floating-Point Fused Multiply-Add with Mixed-Precision Support. *IEEE Trans. Comput.* 68, 7 (2019), 1035–1048. doi:10.1109/TC.2019.2895031